

Week 1 — Data Foundations

Introduction

This week kicks off a machine learning project for a city-wide bike sharing company. The company operates two rental modes — advance bookings and direct pickups — and needs to reliably forecast daily bike demand to allocate resources across the city.

Before any forecasting model can be built, the underlying data must be explored, cleaned, and structured. This week focuses entirely on that foundation: understanding the available data sources, performing the necessary transformations, and organizing the work into a reusable preprocessing pipeline.

A key idea here is that preprocessing should not be treated as a one-time activity. In practice, new data arrives regularly, team members need to reproduce each other's results, and the steps that produce a dataset must be auditable. A well-structured pipeline — where each step has clear inputs, outputs, and dependencies — makes all of this possible. It also reduces the kind of silent mistakes that tend to creep in when data preparation is done in an ad hoc way.

You will work with four data sources: booked rentals, direct pickups, historical weather data, and a holiday calendar. These will be combined into a single structured dataset ready for machine learning in the next stage of the project.

Goal

Build a data preprocessing pipeline that loads, combines, and prepares all available data sources into a single clean dataset suitable for machine learning model training.

The pipeline should not be a one-off script — it must be structured so that each step has clear inputs and outputs, can be re-run when new data arrives, and can be monitored through a graphical interface.

Scope

This week's work covers the following pipeline steps:

- **Load and combine operational data** — Load booked rental and direct pickup records from the database. Aggregate individual records to hourly totals and merge them into a single rental dataset.
- **Derive time-based features** — Engineer additional features from date and timestamp fields (e.g. hour of day, day of week, month) to help the model capture temporal patterns.
- **Enrich with weather data** — Join the historical weather CSV (temperature, humidity, conditions) to the rental dataset on the matching timestamp.
- **Add holiday information** — Merge the manually curated holiday calendar CSV to flag days where travel patterns may differ from a typical workday.
- **Structure work as a Dagster pipeline** — Translate the processing steps from your exploratory notebook into a Dagster asset graph with explicit dependencies, IO managers, and resources.

Expected Outcomes

By the end of this week you will have delivered:

- **A Jupyter notebook** documenting your data exploration and the reasoning behind each transformation choice.
- **A working Dagster pipeline** that materializes all assets without errors, with data assets implemented in the `assets/` directory and a clear separation of responsibilities across assets, resources, and IO managers.
- **A final prepared CSV file** written to disk as the output of the last pipeline asset, ready for model training in Week 2.
- **A GitHub repository** with clean, linted code (Ruff) and NumPy-style docstrings on non-trivial functions, submitted to your track mentor the evening before the code review.

Bonus Part

Only attempt this after fully completing the main assignment.

In real-world pipelines, data is rarely written to local disk — it is pushed to distributed object storage (e.g. AWS S3, Google Cloud Storage) to enable reproducibility, reliability, and collaboration. This bonus simulates that setup locally.

- **Spin up a local RustFS instance** (an S3-compatible object store) using the Docker Quick Start in the RustFS GitHub readme, then create a bucket.
- **Extend your IO logic to support S3 URIs** (e.g. `s3://my-bucket/myfile.csv`). Local paths and S3 URIs should both work — use an environment variable in your Dagster definitions to toggle between them.
- **Explore Python storage clients** — you can use any S3 client; `fsspec` is a good option if you prefer a POSIX-style file API.
- **Optional shortcut** — there is a lesser-known feature of pandas' CSV reader that can significantly simplify steps 2 and 3. See if you can find it.

Resources & Constraints

Required Tools & Libraries

- **IDE:** Visual Studio Code with the Python extension, Jupyter extension, and Ruff (linter/formatter)
- **Dependency management:** `uv` — use it to create a virtual environment and install packages from the project spec
- **Data exploration:** Jupyter Notebooks
- **Data manipulation:** `pandas`
- **Pipeline framework:** Dagster — use assets for processing steps, resources for configuration, and IO managers for data persistence

Helpful Links

- **Dagster docs** — docs.dagster.io
- **pandas docs** — pandas.pydata.org/docs

- **uv docs** — docs.astral.sh/uv
- **Ruff docs** — docs.astral.sh/ruff
- **RustFS GitHub (bonus)** — github.com/rustfs/rustfs

Hints & Pitfalls to Avoid

- **Start in a notebook, not in Dagster.** Verify every transformation step interactively before building the pipeline. Moving to Dagster before the logic is stable wastes time.
- **Don't skip the aggregation step.** Raw records must be aggregated to hourly totals before any join — merging at record level will produce incorrect results.
- **Keep responsibilities separate in Dagster.** Assets handle data processing logic. Resources handle configuration and external connections. IO managers handle reading and writing data.
- **Watch for timezone inconsistencies** when joining weather data to rental timestamps — mismatched offsets are a common source of silent errors.
- **Run Ruff before committing.** Formatting and linting are part of the acceptance criteria — leave enough time to fix any issues before submission.
- **Write docstrings as you go**, not at the end. NumPy-format docstrings are required on all non-trivial functions.